Lesson 2
コントローラ入力
Controller Input

StampFly Workshop

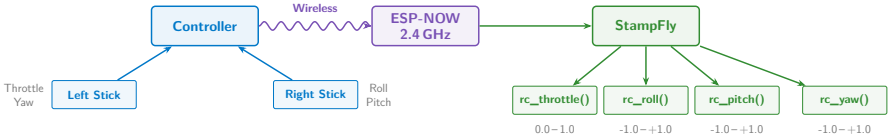
今日のゴール / Today's Goal

目標: スティック値を変数に読み取り、演算でモータを個別制御する

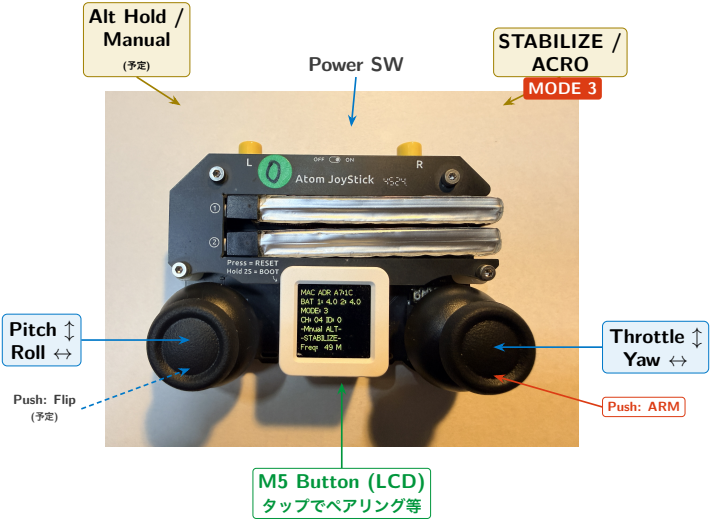
- ESP-NOW 無線通信の仕組みを理解
- チャンネル設定で他の受講生との混信を回避
- 変数と四則演算でスティック → モータ Duty を計算
- オープンループ手動操縦の限界を体感

ESP-NOW 通信フロー / Communication Flow

Controller → ESP-NOW → StampFly API



コントローラ各部 / Controller Layout (MODE 3)



コントローラ API

関数	説明	値域
`set_channel(ch)`	WiFi チャンネル設定	1, 6, 11
`rc_throttle()`	スロットル	0.0 – 1.0
`rc_roll()`	ロール	-1.0 – +1.0
`rc_pitch()`	ピッチ	-1.0 – +1.0
`rc_yaw()`	ヨー	-1.0 – +1.0
`is_armed()`	ARM 状態	true / false
`motor_set_duty(id, v)`	モータ個別制御	id=1–4, v=0.0–1.0

ペアリング手順 / Pairing

Step	操作
1	コントローラの **M5 ボタンを押しながら電源 ON** → LCD に “Pairing mode...” 表示
2	StampFly のボタンを **3 秒長押し** → 青 LED 高速点滅 + ビープ音
3	ペアリング完了 → ビープ音が鳴り、次回から自動接続

注意

- 教室では**一組ずつ**ペアリングする（ブロードキャスト通信のため近くの機体と干渉する可能性）
- うまくいかない場合: 両方を再起動して Step 1 からやり直す

チャンネル設定 / Channel Setting

教室で複数の StampFly を同時に飛ばすと、同じチャンネル同士で**混信**が起きる。チャンネルを分けて回避する。

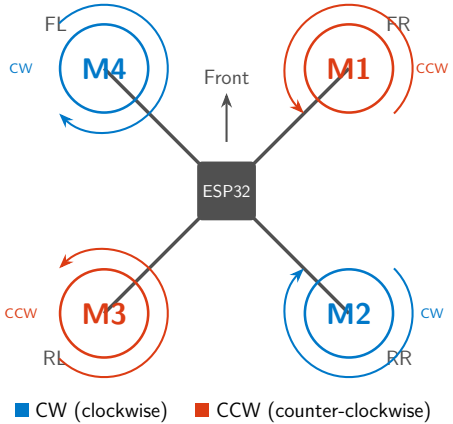
Ch	割り当て例
1	グループ A（緑）
6	グループ B（黄）
11	グループ C（赤）

設定方法

`set_channel(ch)` を `setup()` 内で呼ぶ

機体とコントローラで同じ番号にすること！

モータ配置とミキシング / Motor Layout & Mixing



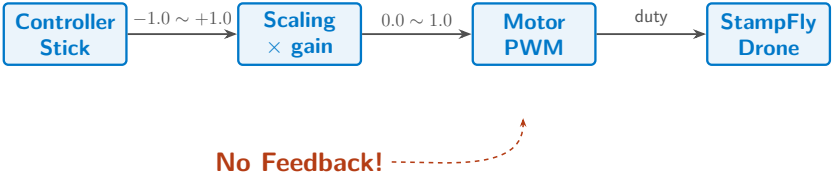
Motor	T	Roll	Pitch	Yaw
M1 FR	+	−	+	+
M2 RR	+	−	−	−
M3 RL	+	+	−	+
M4 FL	+	+	+	−

直感的に理解する

Pitch+ → 前傾 → 前 (M1,M4) 強 + 後 (M2,M3) 弱
Roll+ → 右傾 → 右 (M1,M2) 弱 + 左 (M3,M4) 強

オープンループ制御 / Open-Loop Control

Open-Loop Control



問題点

フィードバックがないため、外乱（風・重心ずれ）に対応できない！
→ Lesson 5–6 で PID 制御を導入して解決

実習: 手動ミキシング (1/2) / Hands-on: Setup

```cpp
#include "workshop_api.hpp"
static uint32_t tick = 0;

void setup() {
    ws::print("L2: Open-Loop Control");
    ws::arm();                  // Enable motor output
    ws::set_channel(1);         // 1, 6, or 11
}
```

user_code.cpp — setup

チャンネル番号

講師が指定した番号（1 / 6 / 11）に書き換えること！
コントローラも同じチャンネルに設定する

実習: 手動ミキシング (2/2) / Hands-on: Loop

```cpp
void loop_400Hz(float dt) {
    tick++;
    float t = ws::rc_throttle();
    float r = ws::rc_roll()  * 0.3f;
    float p = ws::rc_pitch() * 0.3f;
    float y = ws::rc_yaw()   * 0.3f;
    ws::motor_set_duty(1, t - r + p + y); // FR
    ws::motor_set_duty(2, t - r - p - y); // RR
    ws::motor_set_duty(3, t + r - p + y); // RL
    ws::motor_set_duty(4, t + r + p - y); // FL
    if (tick % 80 == 0)
        ws::print("T=%.2f R=%.2f P=%.2f Y=%.2f",
            t, r, p, y);
}
```

user_code.cpp — loop

チェックポイント / Checkpoint

確認事項

- ☐ 指定チャンネルで混信なく通信できた
- ☐ コントローラとペアリングできた
- ☐ スロットルで全モータが均等に回る
- ☐ ピッチスティックで前後モータの回転差が出る

次のレッスン

Lesson 3: LED 制御 — システム状態を LED で可視化